

Reviewer Pack — Edge Triangle-Corrected Forman-Ricci Sandwich for F^*_v

Entry dca566675540 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-29 10:38:02 UTC

Edge Triangle-Corrected Forman-Ricci Sandwich for F^*_v

Entry ID: dca566675540 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-29 10:38:02 UTC

1. Conjecture

Field A (mathematical branch): Combinatorial Bochner (triangle-corrected Forman-Ricci curvature) **Field B** (complexity object): Monotone Boolean functions (k-clique minterm DNF F^*_v of K_v)

Statement:

For $v \in \{10,16,20,24\}$ and $k = \lceil \log_2 v \rceil$, let F_v denote the k-clique minterm DNF on the complete graph K_v whose minterms are the $C(v,k)$ k-subsets of $[v]$. Build the (k-1)-overlap graph G_v : vertex set = the minterms, with an edge between S and T iff $|S \cap T| = k-1$; let $N_{G_v}(\cdot)$ denote G_v -neighborhoods. Define the unsigned combinatorial Bochner invariant $\mu(F_v) := -\min \text{ over edges } e=(S,T) \text{ of } G_v \text{ of } [4 - \deg_{G_v}(S) - \deg_{G_v}(T) + 3 \cdot |N_{G_v}(S) \cap N_{G_v}(T)|]$. Conjecture: $v/4 \leq \mu(F^*_v) \leq 2vk$ for every $v \in \{10,16,20,24\}$; a single (v, edge e) pair violating either inequality refutes it.

Rationale (proposer's reasoning):

This step advances SC5 by simultaneously witnessing the desired lower bound $v/4$ and pinning the upper scale at $O(vk) \approx O(v \log v)$, turning the open SC5 question into a quantitative sandwich. The triangle correction is the natural Bochner-style refinement of bare Forman curvature and is the discrete analogue of Ricci lower bounds via simplex multiplicities; on the highly transitive G_v it should be constant across edges (a strong, falsifiable prediction). If even one edge breaks regularity or pushes μ below $v/4$, SC5 is decisively refuted on F^*_v .

Taxonomy category: forman_ricci_dnf (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): a81c838e26aff17e

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For each $v \in \{10,16,20,24\}$ with $k=\lceil \log_2 v \rceil$, sample 30 seeds drawing (S,T) edges in G_v and compute $\mu_e = -(4 - \deg(S) - \deg(T) + 3 \cdot |N(S) \cap N(T)|)$. Let $\mu(F^*_v) = \min$ over the 30 sampled edges. **SUPPORTED** iff $v/4 \leq \mu(F^*_v) \leq 2vk$ for all four v; **FALSIFIED** iff any (v, seed) yields $\mu_e < v/4$ or $\mu_e > 2vk$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.90	SAFE	SAFE
ALGEBRIZATION	SAFE	0.92	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Forman-Ricci curvature triangle correction Johnson graph - augmented Forman Ricci curvature clique complex monotone Boolean - combinatorial Bochner Forman curvature k-clique DNF overlap graph

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    v_values = [10, 16, 20, 24]
    results = []

    for v in v_values:
        k = math.ceil(math.log2(v))
        minterms = list(itertools.combinations(range(1, v + 1), k))
        n = len(minterms)

        if n < 30:
            return {
                "metric_name": " $\mu(F*_v)$ ",
                "metric_value": None,
                "instances_tested": 0,
                "n_max": n,
                "conjecture_holds": False,
                "counterexample": "not_enough_minterms"
            }

        random.seed(seed + v)
        samples = [(random.choice(minterms), random.choice([x for x in minterms if len(set(x) - se

        min_mu_e = float('inf')
        counterexample = ""

        for S, T in samples:
            deg_S = k * (v - k)
```

```

deg_T = k * (v - k)

N_S = set()
N_T = set()

for S_prime in itertools.combinations(S, k-1):
    if len(set(S_prime) | {S[0]}) == k:
        N_S.add(tuple(sorted(S_prime + (S[0],))))

for T_prime in itertools.combinations(T, k-1):
    if len(set(T_prime) | {T[0]}) == k:
        N_T.add(tuple(sorted(T_prime + (T[0],))))

intersection_size = len(N_S & N_T)
mu_e = -(4 - deg_S - deg_T + 3 * intersection_size)
min_mu_e = min(min_mu_e, mu_e)

if mu_e < v / 4 or mu_e > 2 * v * k:
    counterexample = f"(v={v}, S={S}, T={T},  $\mu_e$ ={mu_e})"

results.append({
    "metric_name": " $\mu(F*_v)$ ",
    "metric_value": min_mu_e,
    "instances_tested": len(samples),
    "n_max": n,
    "conjecture_holds": v / 4 <= min_mu_e <= 2 * v * k,
    "counterexample": counterexample
})

return {
    "metric_name": " $\mu(F*_v)$ ",
    "metric_value": sum(r["metric_value"] for r in results) / len(results),
    "instances_tested": sum(r["instances_tested"] for r in results),
    "n_max": max(r["n_max"] for r in results),
    "conjecture_holds": all(r["conjecture_holds"] for r in results),
    "counterexample": next((r["counterexample"] for r in results if r["counterexample"]), "")
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43]

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std=0.0 support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(i for i, r in enumerate(results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{results[first_failing_seed]['counterexample']}")
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9930871c.py", line 87, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9930871c.py", line 38, in run_trial
    samples = [(random.choice(minterms), random.choice([x for x in minterms if len(set(x) -
set(S)) == 1 and len(set(S) | {x[0]}) == k])) for _ in range(30)]
                                                    ^
UnboundLocalError: cannot access local variable 'S' where it is not associated with a value
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with an UnboundLocalError before producing any data, so neither the support nor the falsification condition can be evaluated. Per the rules, a crashed test yields an INCONCLUSIVE verdict. | next: Fix the buggy edge-sampling comprehension (the inner generator references S before it is bound) by first drawing $S = \text{random.choice}(\text{minterms})$, then sampling T from minterms with $|S \cap T| = k-1$, and re-run for $v \in \{10, 16, 20, 24\}$.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	157982
2	propose	claude_max	opus	0	0	136230
3	preregistration	claude_max	opus	0	0	5707
4	novelty	claude_max	opus	0	0	4364
5	novelty	claude_max	opus	0	0	5709
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17963
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17488
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11070
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12011
10	judge	claude_max	opus	0	0	4835

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 373360 ms total latency. Provider mix: {'claude_max': 6, 'ollama_remote': 4}

(full prompt+response transcripts available in research/audit/dca566675540.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/dca566675540.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/dca566675540.tar.gz (if generated)